GoldAlert SG — Technical Documentation

> *Last updated: 2026-03-26*

Table of Contents

1. Overview
2. System Architecture
3. Data Flow
4. Scheduler — GitHub Actions
5. Scraper — gold_bot.py
   - Retry and Deadline Model
   - Mustafa Jewellery
   - Malabar Gold SG
   - Joyalukkas SG
   - GRT Jewels SG
   - Failure Fallback
6. Price Chart Generation
7. Email Notification System
8. Airtable Data Model
9. Web Frontend — Next.js
10. API Routes
    - POST /api/subscribe
    - GET /api/metrics
    - GET/POST /api/visitors
    - POST /api/unsubscribe/request
    - POST /api/unsubscribe/confirm
11. Environment Variables
12. Key Engineering Challenges and Solutions
13. Reliability Mechanisms
14. Local Development Setup
15. Glossary

---

1. Overview

GoldAlert SG is a fully serverless system that monitors live gold prices across four Singapore jewellers and sends email alerts to subscribers when prices are scraped. It targets buyers of 22k (916 purity) and 24k (999 purity) gold who want to time their purchases around price movements.

The problem it solves: Gold prices in Singapore fluctuate throughout the day and vary between shops by meaningful amounts. Manually checking four separate jeweller websites is tedious. GoldAlert SG automates this, delivering a single consolidated email — with a price history chart and trend arrows — seven times per day.

Key properties:

- Zero infrastructure cost — no servers, no databases to manage
- Fully automated — scraping, alerting, and subscriber management are end-to-end automated
- Fault-tolerant — individual scraper failures do not stop the email from sending
- No user account required — subscribe and unsubscribe with only an email address

---

2. System Architecture

```
┌──────────────────────────────────────────────────────────┐
│                    GitHub Actions (Cron)                  |
│        7×/day · UTC 00:05, 02:05, 04:05, 06:05,          |
│                  08:05, 10:05, 12:05                      |
└──────────────────────────┬───────────────────────────────┘
                           | triggers
                           ▼
┌──────────────────────────────────────────────────────────┐
│                 scraper/gold_bot.py (Python 3.11)        |
│                                                          |
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐ |
│  |   Mustafa    |  |   Malabar    |  |   Joyalukkas     | |
│  | BeautifulSoup|  | BeautifulSoup|  |   GraphQL API    | |
│  └──────────────┘  └──────────────┘  └──────────────────┘ |
│  ┌──────────────┐                                         |
│  |  GRT Jewels  |                                         |
│  | cloudscraper |                                         |
│  | +BeautifulSoup                                         |
│  └──────────────┘                                         |
│                          |                                |
│             each shop: up to 3 attempts                   |
│             45s total deadline per shop                   |
└──────────┬──────────────────┬──────────────────────────┘
           | save prices      | read subscribers + last prices
           ▼                  ▼
┌──────────────────────────────────────────────────────────┐
│                         Airtable                         |
│   tables: prices · subscribers · visitors · otps         |
└──────────────────────────┬───────────────────────────────┘
                           | read all prices for chart
                           | read subscriber emails
                           ▼
┌──────────────────────────────────────────────────────────┐
│              Gmail SMTP (smtp.gmail.com:465)             |
```

```
│   HTML email + inline PNG chart → each subscriber          │
└──────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────┐
│            Next.js 14 App Router (Vercel)                  │
│   Landing page · Subscribe form · Unsubscribe (OTP flow)   │
│   API routes: /subscribe · /metrics · /visitors            │
│               /unsubscribe/request · /unsubscribe/confirm  │
└──────────────────────────────────────────────────────────┘
```

The two systems (scraper and web app) are decoupled. They share only Airtable as a common data store. The scraper never calls the web app, and the web app never triggers the scraper.

---

3. Data Flow

Scraping and alert run (triggered by GitHub Actions)

```
1.  Scrape all 4 shops (in sequence, each with its own retry loop)
2.  Fetch last known price per shop from Airtable (for trend arrows)
3.  Build email body — one section per shop, with ↑/↓ indicators
4.  Fetch full price history from Airtable (for chart)
5.  Generate dual-subplot PNG chart (22k top, 24k bottom)
6.  Fetch subscriber list from Airtable
7.  Send one HTML email per subscriber via Gmail SMTP
        └── plain text fallback + HTML body + inline CID chart image
8.  Save scraped prices to Airtable (only for shops where status=OK)
```

> *Note: Step 8 (saving prices) happens **after** sending the email. This is intentional — the chart is generated from historical data, not including the current run's prices. The current prices appear in the email text body only.*

Subscription flow (triggered by user on web app)

```
User enters email → POST /api/subscribe
  → Airtable duplicate check (filterByFormula)
  → If duplicate: return HTTP 409
  → If new: create record in `subscribers` table → return HTTP 200
```

Unsubscription flow (OTP-verified)

```
User enters email → POST /api/unsubscribe/request
  → Look up email in `subscribers` table
  → Generate 6-digit OTP
  → Delete any existing OTP for this email from `otps` table
```

```
  → Save new OTP to `otps` table
  → Send OTP via Gmail (nodemailer)
  → Always return HTTP 200 (email existence not revealed)

User enters OTP → POST /api/unsubscribe/confirm
  → Look up matching (email, otp) record in `otps` table
  → Check Airtable createdTime against 10-minute TTL
  → If expired: delete OTP record, return HTTP 400
  → If valid: delete subscriber record + delete OTP record → return HTTP 200
```

---

4. Scheduler — GitHub Actions

File: `.github/workflows/goldrates.yml`

The workflow runs on a UTC cron schedule that maps to Singapore business hours (SGT = UTC+8):

UTC time	SGT time
00:05	08:05
02:05	10:05
04:05	12:05
06:05	14:05
08:05	16:05
10:05	18:05
12:05	20:05

This gives **7 runs per day**, covering 8:05am to 8:05pm SGT.

Cron expression: `"5 0,2,4,6,8,10,12 * * *"`

The workflow steps are minimal by design:

```yaml
- uses: actions/checkout@v4
- uses: actions/setup-python@v5
  with:
    python-version: "3.11"
- run: pip install -r scraper/requirements.txt
- run: python scraper/gold_bot.py
  env:
    AIRTABLE_API_KEY: ${{ secrets.AIRTABLE_API_KEY }}
    AIRTABLE_BASE_ID: ${{ secrets.AIRTABLE_BASE_ID }}
    GMAIL_USER: ${{ secrets.GMAIL_USER }}
    GMAIL_APP_PASSWORD: ${{ secrets.GMAIL_APP_PASSWORD }}
```

Why `:05` past the hour, not `:00`? GitHub's shared cron queue is heavily congested at the top of every hour. Scheduling at `:05` past the hour significantly reduces queue wait time and keeps runs close to their intended start time. See Section 12 for the full history of this decision.

The workflow also supports `workflow_dispatch`, allowing a manual trigger from the GitHub Actions UI at any time without modifying the schedule.

---

5. Scraper — gold_bot.py

File: `scraper/gold_bot.py`

Dependencies (`scraper/requirements.txt`):

Package	Purpose
`requests`	HTTP fetching for Mustafa, Malabar, Joyalukkas
`cloudscraper`	Bot-bypass HTTP client for GRT (mimics Chrome TLS fingerprint)
`beautifulsoup4`	HTML parsing for Mustafa, Malabar, GRT
`pytz`	SGT timezone handling
`pyairtable`	Airtable read/write operations
`python-dotenv`	Local `.env` loading (skipped in CI where env vars are injected)
`matplotlib`	Headless PNG chart generation

The scraper uses a simple environment detection pattern: if `AIRTABLE_API_KEY` is not set as an environment variable (which it always will be in GitHub Actions), it falls back to loading a `.env` file from the project root. This means no code changes are needed between local development and CI.

5.1 Retry and Deadline Model

All scrapers (except where noted) use the same retry model implemented in `scrape_with_retry()` and equivalent per-shop functions.

Parameters:

- `MAX_ATTEMPTS = 3`
- `TOTAL_DEADLINE_SECONDS = 45` (15 seconds per attempt on average)

How it works:

```
start = time.time()

for attempt in 1..3:
    remaining = 45 - (time.time() - start)
    if remaining <= 0: break
```

```
try:
    html = fetch(url, timeout=remaining)   ← remaining time is the HTTP timeout
    parse and return result
except:
    backoff = 0.5 * 2^(attempt-1)         ← 0.5s, 1.0s, 2.0s
    sleep(backoff) if time allows
```

The `remaining` time is passed directly as the HTTP `timeout` parameter. This means each attempt gets the full remaining budget, not a fixed slice. If the first attempt takes 14 seconds and fails, the second attempt still has ~31 seconds. This is the correct model for flaky-network retries.

> ***Important:*** *The original implementation used a fixed 15-second total deadline, which meant a single slow failed attempt consumed the entire budget and left no time for retries. The deadline was increased to 45 seconds to give all three attempts a realistic chance. See Section 12.*

5.2 Mustafa Jewellery

URL: `https://mustafajewellery.com/`

Method: `requests.get` + BeautifulSoup

How it works: Mustafa's website exposes prices directly in the HTML as elements with stable IDs. No JavaScript execution is needed.

Target elements:

Element ID	Content
`#22k_price1`	22k (916) price as a plain number, e.g. `204.40`
`#24k_price1`	24k (999) price as a plain number, e.g. `222.00`
`#date_update_gold`	Source last-updated date
`#time_updates_gold`	Source last-updated time

Parser function: `parse_mustafa_rates(html)`

After extraction, prices are validated with `is_numberish()` to confirm they can be parsed as floats. If the price text is empty or non-numeric, a `ValueError` is raised and the retry loop catches it.

5.3 Malabar Gold SG

URL: `https://www.malabargoldanddiamonds.com/stores/singapore`

Method: `requests.get` + BeautifulSoup

How it works: Malabar's page uses country-specific element IDs. Singapore's country code on their platform is `85`, so the IDs follow the pattern `price22kt_85`, `price24kt_85`.

Target elements:

Element ID	Content
`#price22kt_85`	22k price in format `179.00 SGD`
`#price24kt_85`	24k price in format `196.00 SGD`
`#updatedtime_85`	Source last-updated timestamp

Price cleaning: The raw text includes a `SGD` currency suffix. This is stripped using `re.sub(r"[^\d.]", "", raw)` before numeric validation.

5.4 Joyalukkas SG (GraphQL)

Endpoint: `https://www.joyalukkas.com/graphql`

Method: `requests.post` with JSON body — this is a direct API call, not HTML scraping.

Why GraphQL instead of scraping? Joyalukkas loads its price data dynamically. Rather than requiring a headless browser to execute JavaScript, they happen to expose a public GraphQL endpoint that returns structured price data directly.

Request structure:

```
query   = "{getgoldrates{metal_rate_time Data{GOLD_22KT_RATE GOLD_24KT_RATE}}}"
headers = {
    "Content-Type": "application/json",
    "Accept":       "application/json",
    "Store":        "sg",     # critical: selects Singapore store and SGD pricing
}
```

The `Store: sg` header is mandatory. Without it, the API returns prices for a different region in a different currency.

Response path:

```
response.data.getgoldrates.Data[0].GOLD_22KT_RATE  → "169.00"
response.data.getgoldrates.Data[0].GOLD_24KT_RATE  → "184.00"
response.data.getgoldrates.metal_rate_time          → last updated timestamp
```

The price values are returned as strings and are `.strip()` ped before numeric validation.

5.5 GRT Jewels SG

URL: `https://www.grtjewels.com/asia/`

Method: `cloudscraper` + BeautifulSoup

Why cloudscraper? GRT's website uses Cloudflare or a similar WAF (Web Application Firewall) that blocks requests with a simple user-agent string. `cloudscraper` works around this by:

- Mimicking a real browser's TLS fingerprint (cipher suite order, extensions)
- Including a complete set of browser-like HTTP headers
- Handling JS challenge cookies automatically

The scraper is configured to emulate Chrome on Windows:

```
scraper = cloudscraper.create_scraper(
    browser={"browser": "chrome", "platform": "windows", "mobile": False}
)
```

The dropdown problem: GRT's "Today's Rate" prices are inside a CSS hover dropdown menu. The menu is invisible until a user hovers over the nav item. However, the price data **is present in the initial HTML** — it is the CSS that hides it, not JavaScript that injects it. This means BeautifulSoup can parse the prices without any JavaScript execution.

CSS navigation path:

```
<li class="menu-item menu-item-has-children">
  <a href="#">Today's Rate ▾</a>
  <ul class="sub-menu sf-sub-indicator">
    <li><a href="#">GOLD - 22KT -  1. g  -  SGD  $ 169.90</a></li>
    <li><a href="#">GOLD - 24KT -  1. g  - SGD $ 184.60</a></li>
  </ul>
</li>
```

Parser logic in `parse_grt_rates(html)`:

1. Find all `<li class="menu-item-has-children">` elements
2. Among those, find the one whose direct `<a>` text contains both "today" and "rate" (case-insensitive)
3. From that `<li>`, find the `<ul class="sub-menu">` child
4. Extract text from all `<a>` tags within the sub-menu
5. For each text, identify 22KT vs 24KT via regex, then extract the price

Price extraction in `_extract_price_from_text(text)`:

The function uses a two-pass approach:

- **Primary:** match the number that follows `SGD` and optional `$`, e.g. `SGD $ 169.90` → `169.90`
- **Fallback:** scan for any number in the plausible per-gram SGD range of 80–600

The fallback ensures the parser remains functional even if GRT reformats the currency symbol or label, as long as the price number remains in the expected range.

5.6 Failure Fallback with Old Rates

When a scraper exhausts all retries and returns `status: FAILED`, the email is not silently omitted for that shop. Instead, `build_shop_section()` fetches the last known price from Airtable using `get_last_prices(shop)` and displays it with a clear warning label:

```
🏪 GRT Jewels SG
22k (916): S$169.90 (Old rate — server error)
24k (999): S$184.60 (Old rate — server error)

Status: FAILED — <error detail>
Check latest: https://www.grtjewels.com/asia/
```

If no historical price exists either, the section shows only the failure status and error message.

This approach ensures subscribers always receive a complete email and can click the shop link to check manually.

---

6. Price Chart Generation

Function: `generate_price_chart(history: list) -> bytes | None`

The chart is a dual-subplot PNG rendered headlessly using matplotlib with the `Agg` backend (which does not require a display). It is generated fresh on every run using the full historical price dataset fetched from Airtable.

Chart specifications:

Property	Value
Backend	`matplotlib.use("Agg")` — headless, no display
Size	12 × 7 inches at 150 DPI
Layout	2 subplots sharing the x-axis: 22K on top, 24K on bottom
Background	Near-black `#0d0d0d` (figure), `#161616` (axes)
Title color	Gold `#c8a84b`
Grid	Dashed, `#222222` , 60% opacity
X-axis	SGT datetime, `"DD Mon"` format, auto-spaced

Shop colors:

Shop	Color	Hex
Mustafa Jewellery	Light blue	`#4FC3F7`
Malabar Gold SG	Light green	`#81C784`
Joyalukkas SG	Amber	`#FFB74D`
GRT Jewels SG	Lavender	`#CE93D8`

Output: The chart is rendered to a `BytesIO` buffer and returned as raw PNG bytes. If the price history list is empty, the function returns `None` and the email is sent without a chart.

---

7. Email Notification System

Function: `send_email_to_all(message: str, chart_bytes: bytes | None)`

Transport: Gmail SMTP over SSL on port 465, using a Google App Password (not the account password). The App Password must be generated separately in Google Account security settings with 2-Step Verification enabled.

Email structure: Each email is a `MIMEMultipart("related")` message, which is the correct MIME type for embedding inline images. The structure is:

```
MIMEMultipart("related")
├── MIMEMultipart("alternative")
│   ├── MIMEText(plain_text, "plain")    ← fallback for clients that don't render HTML
│   └── MIMEText(html_body, "html")      ← primary: dark-themed HTML with chart
└── MIMEImage(chart_png)                 ← inline image with Content-ID: <goldchart>
```

The HTML body references the chart via `<img src="cid:goldchart">`. This is **inline embedding** — the image is part of the email, not a remote URL or attachment. Email clients display it without needing internet access to load it, and it avoids spam filters that flag remote tracking pixels.

Subject line format: `📊 Gold Prices · 26 Mar 10:05 AM` — uses the SGT time of the run.

Per-subscriber loop: A 1-second sleep between emails (`time.sleep(1)`) prevents Gmail rate limiting when the subscriber list grows.

Trend indicators: Price sections include `↑` or `↓` symbols when the scraped price differs from the previous Airtable record for that shop. These are computed in `build_shop_section()` using simple float comparison.

---

8. Airtable Data Model

Airtable serves as the sole data store. There are four tables:

`subscribers`

Field	Type	Notes
`email`	Single line text	Primary key (enforced by app logic, not Airtable)

Duplicate protection is enforced at the API layer using a `filterByFormula` check before insert. Airtable does not enforce uniqueness natively.

`prices`

Field	Type	Notes
`shop`	Single line text	One of the four shop name strings
`price_22k_916`	Number (or text)	Per-gram SGD price for 22k gold
`price_24k_999`	Number (or text)	Per-gram SGD price for 24k gold

`createdTime`	Auto (Airtable)	Used for chart x-axis and OTP expiry checking

One record is created per shop per successful run. A failed scrape produces no record for that shop.

`visitors`

Field	Type	Notes
`count`	Number	Incremental count of unique sessions

A single record is maintained. The API route reads it and adds `VISITOR_SEED = 200` before returning the display value. The seed provides a non-zero starting count.

`otps`

Field	Type	Notes
`email`	Single line text	The subscriber email requesting unsubscription
`otp`	Single line text	6-digit numeric code
`createdTime`	Auto (Airtable)	Used to enforce 10-minute TTL

Existing OTP records for an email are deleted before a new one is saved, ensuring only one valid code exists per email at any time.

---

9. Web Frontend — Next.js

Location: `web/`

Framework: Next.js 14 with the App Router. Deployed to Vercel with root directory set to `web/`.

Fonts (loaded via `next/font/google`, no external requests at runtime):

Variable	Font	Use
`--font-cormorant`	Cormorant Garamond	Display headings (editorial, serif)
`--font-outfit`	Outfit	Body text (modern, clean)
`--font-mono`	JetBrains Mono	Numbers and metrics (monospaced)

Page sections (`web/app/page.tsx`):

1. **`GoldCanvas`** — Full-page canvas animation: 130 gold dust particles drifting upward with sinusoidal drift, plus 7 large radial gradient glow orbs drifting slowly. Rendered on a `requestAnimationFrame` loop. Cleaned up on unmount with `cancelAnimationFrame`. Uses `aria-hidden="true"` to be invisible to screen readers.
2. **`VisitorBadge`** — Displays the visitor count. Uses `sessionStorage` with key `ga_visited` to distinguish first visits from return visits within the same browser session. First visit: `POST /api/visitors`

(increments then reads). Return visit: `GET /api/visitors` (reads only). This prevents one user refreshing the page from inflating the count.

3. **`SubscribeForm`** — Client-side form with five states: `idle`, `loading`, `success`, `error`, `duplicate`. On success, fires a custom `metrics-refresh` window event to trigger an immediate metrics update without waiting for the 30-second poll interval.
4. **`LiveMetrics`** — Polls `GET /api/metrics` every 30 seconds and listens for the `metrics-refresh` event. Displays subscriber count and total alerts sent.
5. **Static sections** — Features grid, testimonials, how-it-works steps, value proposition, and CTA — all server-rendered, no dynamic data.
6. **Scroll reveal** — A single `IntersectionObserver` adds the `.visible` class to `.reveal` elements as they enter the viewport at a 12% threshold. Pure CSS handles the transition. The observer is disconnected on component unmount.

Unsubscribe page (`web/app/unsubscribe/page.tsx`):

A three-step client component:

- **`email`** — Input email address, submit to request OTP
- **`otp`** — Input 6-digit code (numeric-only enforced by `.replace(/\D/g, '')`)
- **`done`** — Confirmation screen with link back to home

The OTP step message does not confirm whether the email is subscribed ("If [email] is subscribed, a code was sent"), preventing enumeration of the subscriber list.

---

10. API Routes

All routes are Next.js App Router Route Handlers in `web/app/api/`.

10.1 POST /api/subscribe

File: `web/app/api/subscribe/route.ts`

Method	`POST`
Body	`{ "email": "user@example.com" }`
Success	`200 { "success": true }`
Duplicate	`409 { "error": "already_subscribed" }`
Invalid email	`400 { "error": "Invalid email address" }`
Env missing	`503 { "error": "Service unavailable" }`

Logic:

1. Validate email format (must contain `@` and `.`)

2. Query Airtable `subscribers` with `filterByFormula=({email}="<email>")` to detect duplicates
3. If found: return 409
4. If not found: `POST` to Airtable to create the subscriber record

10.2 GET /api/metrics

File: `web/app/api/metrics/route.ts`

Method	`GET`
Response	`{ "subscribers": 42, "notifications": 1347 }`

Subscribers: Direct count of records in the `subscribers` table.

Notifications formula:

```
runs          = floor(price_records / SHOP_COUNT)       // SHOP_COUNT = 4
notifications = NOTIFICATIONS_BASE + (runs × subscribers)  // NOTIFICATIONS_BASE =
1000
```

This derives email sends from structural data rather than storing a separate counter. Each bot run creates exactly one price record per shop (4 total), so dividing total price records by 4 gives total successful runs. Multiplying by current subscriber count approximates total emails ever sent. The `NOTIFICATIONS_BASE = 1000` seed accounts for historical sends before the current subscriber count was reached.

The route paginates through Airtable using the `offset` token to count all records regardless of table size.

10.3 GET/POST /api/visitors

File: `web/app/api/visitors/route.ts`

Method	Behaviour
`GET`	Returns `{ "count": <VISITOR_SEED + stored_count> }` — read-only
`POST`	Increments the stored count by 1, returns new `{ "count": ... }`

Both methods apply `VISITOR_SEED = 200` to the stored value before returning, ensuring the displayed count starts at 200.

The route performs a PATCH to the single `visitors` Airtable record on POST.

10.4 POST /api/unsubscribe/request

File: `web/app/api/unsubscribe/request/route.ts`

Method	`POST`
Body	`{ "email": "user@example.com" }`

Always returns	`200 { "ok": true }` (regardless of whether email is subscribed)

Logic:

1. Look up email in `subscribers` table
2. If **not found**: return `200` immediately without sending an email (prevents subscriber enumeration)
3. If **found**: generate a random 6-digit OTP (`Math.floor(100000 + Math.random() * 900000)`)
4. Delete all existing OTP records for this email from the `otps` table
5. Save new OTP to `otps` table
6. Send OTP email via nodemailer + Gmail SMTP

10.5 POST /api/unsubscribe/confirm

File: `web/app/api/unsubscribe/confirm/route.ts`

Method	`POST`
Body	`{ "email": "user@example.com", "otp": "123456" }`
Success	`200 { "ok": true }`
Invalid/expired	`400 { "error": "..." }`

Logic:

1. Query `otps` table with `filterByFormula=AND({email}="...", {otp}="...")`
2. If no record: return 400
3. Check `createdTime` against `OTP_TTL_MS = 600000` (10 minutes)
4. If expired: delete OTP record, return 400 with expiry message
5. If valid: delete subscriber record from `subscribers` table, delete OTP record, return 200

---

11. Environment Variables

Scraper (GitHub Actions secrets)

Variable	Description
`AIRTABLE_API_KEY`	Airtable personal access token
`AIRTABLE_BASE_ID`	Airtable base ID (starts with `app`)
`GMAIL_USER`	Gmail address used for sending
`GMAIL_APP_PASSWORD`	16-character Google App Password

Web App (Vercel environment variables)

Variable	Description
`AIRTABLE_API_KEY`	Same token as above
`AIRTABLE_BASE_ID`	Same base ID as above
`GMAIL_USER`	Gmail address (used by unsubscribe request route)
`GMAIL_APP_PASSWORD`	Google App Password (used by unsubscribe request route)

Optional (scraper)

Variable	Default	Description
`SITE_URL`	`https://goldalert-sg.vercel.app`	Unsubscribe link in email footer

---

12. Key Engineering Challenges and Solutions

1. GRT Jewels — Price hidden in hover dropdown

Problem: GRT's "Today's Rate" prices are inside a CSS hover dropdown menu. Using a simple `requests` call with a basic user-agent was blocked by the site's WAF, and even when the HTML was fetched, it was not obvious how to locate the prices since they are visually hidden.

Solution:

- Switched from `requests` to `cloudscraper`, which mimics a real Chrome browser's TLS fingerprint and headers, bypassing WAF blocks
- Confirmed (via DevTools inspection) that the price data is present in the initial HTML as a hidden `<ul class="sub-menu sf-sub-indicator">` inside a `<li class="menu-item-has-children">` — no JavaScript execution needed
- Used exact CSS class selectors to navigate: find the `Today's Rate` nav item by its anchor text, then find the `sub-menu` child
- Text format confirmed: `"GOLD - 22KT - 1. g - SGD $ 169.90"` — parsed with `SGD\s*\$?\s*(\d{2,4}...)` regex

> ***Future risk:*** *If GRT moves price loading to a JavaScript/API call, this parser will stop working. A Playwright-based solution would be needed. The code contains a comment documenting this.*

2. Malabar — Read timeouts killing all retries

Problem: The original `TOTAL_DEADLINE_SECONDS` was set to 15 seconds. Malabar's site is occasionally slow. A single first attempt that took 14 seconds would consume the entire deadline, leaving no time for retries. The result was consistent single-attempt failures that looked like the site was unreachable.

Solution: Increased `TOTAL_DEADLINE_SECONDS` from 15 to 45 seconds. This gives each of the 3 retry attempts a realistic 15-second budget while still ensuring the bot completes well within the GitHub Actions job timeout.

3. SonarCloud blocking Vercel deployments

Problem: A `needs: sonarcloud` gate in the GitHub Actions workflow caused Vercel deployment jobs to wait for (and fail on) SonarCloud scan failures, blocking all production deployments.

Solution: Removed SonarCloud from the workflow entirely. The dependency between code quality scanning and deployment was deemed inappropriate for this project's scale.

4. GitHub Actions cron drift at top of hour

Problem: Crons scheduled at `:00` (e.g. `0 0,2,4,6,8,10,12 * * *`) were consistently delayed 30–60 minutes. GitHub's shared runner queue is heavily loaded at the top of every hour from thousands of repositories scheduling jobs simultaneously.

Solution: Shifted all cron triggers to `:05` past the hour. The window was also adjusted from 9am–11pm SGT to 8am–8pm SGT to better align with peak buying hours and reduce total daily runs from 8 to 7.

5. Joyalukkas — Dynamic page requires API, not scraping

Problem: Joyalukkas loads its price data via JavaScript after page load. A static HTML scrape would return an empty price element.

Solution: Rather than using a headless browser, a DevTools network inspection revealed that Joyalukkas exposes a public GraphQL endpoint at `/graphql`. The price data can be fetched directly with a POST request, providing structured JSON without any HTML parsing. The critical discovery was the `Store: sg` header required to get Singapore-specific SGD prices.

6. Price chart — Headless rendering with inline email embedding

Problem: Email clients do not support externally hosted images in all contexts, and generating images server-side requires a display environment.

Solution:

- `matplotlib.use("Agg")` enables headless rendering with no display dependency — works in GitHub Actions
- Chart is rendered to a `BytesIO` buffer and returned as raw PNG bytes
- Embedded in the email as a CID (Content-ID) inline image: `<img src="cid:goldchart">` with corresponding `MIMEImage` part having `Content-ID: <goldchart>` — the image travels inside the email, not as a remote URL

7. Visitor counter — Avoiding inflation on page refresh

Problem: A naive visitor counter that increments on every page load would be inflated by a single user refreshing the page multiple times.

Solution: `sessionStorage` (not `localStorage`) is used with key `ga_visited`. The first load in a browser session triggers a `POST` to increment the count. Any subsequent load in the same session (including page refresh) triggers only a `GET` to read the count. Closing and reopening the browser starts a new session and counts as a new visit.

8. Notifications counter — Derived, not stored

Problem: Storing a notification counter independently would require the scraper to increment it on every send, adding complexity and a potential failure point.

Solution: The notifications count is **derived** from structural Airtable data:

- Total price records ÷ 4 shops = total successful runs
- Total runs × current subscriber count + 1000 seed = approximate total emails sent

This requires no additional writes and stays consistent with the actual Airtable data.

---

13. Reliability Mechanisms

Mechanism	Implementation
Per-shop retry loop	Up to 3 attempts per shop with exponential backoff (0.5s, 1.0s, 2.0s)
Sliding timeout budget	`remaining = TOTAL_DEADLINE_SECONDS - elapsed` passed as HTTP timeout, so each retry gets maximum available time
Numeric price validation	`is_numberish()` validates all parsed prices before accepting them, preventing garbage values from being stored or emailed
Failure fallback	Failed shops show last known price from Airtable with clear `(Old rate — server error)` label
Independent shop scrapers	Each shop fails independently; one failure does not prevent others from running or the email from sending
Prices saved only on success	`save_prices()` is called only when `result["status"] == "OK"`, preventing null or error values from polluting price history
Duplicate subscription guard	`filterByFormula` check before insert; HTTP 409 returned to the client
OTP expiry	10-minute TTL enforced using Airtable `createdTime`; expired OTP records are deleted before returning the error
Subscriber enumeration protection	`/api/unsubscribe/request` always returns 200 regardless of whether the email is in the subscriber list
Env var guard	All API routes check for `AIRTABLE_API_KEY` / `AIRTABLE_BASE_ID` at request time and return 503 if missing

---

14. Local Development Setup

Prerequisites

- Python 3.11+
- Node.js 18+
- An Airtable base with four tables: `subscribers`, `prices`, `visitors`, `otps`
- A Gmail account with 2-Step Verification and an App Password generated

Scraper

```
# From repo root
pip install -r scraper/requirements.txt

# Create .env in repo root
echo "AIRTABLE_API_KEY=your_token" >> .env
echo "AIRTABLE_BASE_ID=appXXXXXXX" >> .env
echo "GMAIL_USER=you@gmail.com" >> .env
echo "GMAIL_APP_PASSWORD=xxxx xxxx xxxx xxxx" >> .env

# Run once
python scraper/gold_bot.py
```

Web App

```
cd web
npm install

# Create .env.local
cp .env.local.example .env.local
# Edit .env.local: AIRTABLE_API_KEY, AIRTABLE_BASE_ID, GMAIL_USER,
GMAIL_APP_PASSWORD

npm run dev   # http://localhost:3000
```

Vercel Deployment

1. Import the repository at vercel.com/new
2. Set **Root Directory** to `web`
3. Add all four environment variables under Project Settings → Environment Variables
4. Deploy

GitHub Actions Setup

Add four repository secrets under **Settings → Secrets and variables → Actions**:

- `AIRTABLE_API_KEY`
- `AIRTABLE_BASE_ID`
- `GMAIL_USER`
- `GMAIL_APP_PASSWORD`

The workflow runs automatically on the defined cron schedule. Use **Run workflow** in the Actions tab for on-demand testing.

---

15. Glossary

Term	Definition
22k / 916	22-karat gold with 91.6% purity — the most common purity for jewellery in Singapore
24k / 999	24-karat gold with 99.9% purity — investment-grade, close to pure gold
SGT	Singapore Standard Time — UTC+8, no daylight saving time
WAF	Web Application Firewall — server-side bot detection and blocking
cloudscraper	Python library that mimics a real browser's TLS fingerprint to bypass WAF checks
CID (Content-ID)	Email MIME mechanism for embedding images inline inside the email body
App Password	A 16-character Gmail-specific password used in place of the account password for SMTP; requires 2-Step Verification to be enabled
Airtable offset	Pagination token returned by Airtable when a query result exceeds one page (100 records); used to fetch all records across multiple requests
OTP	One-Time Password — a short-lived code sent by email to verify identity before performing a destructive action (unsubscription)
Agg backend	matplotlib's non-interactive renderer that works without a display — required for headless server and CI environments
filterByFormula	Airtable REST API query parameter that filters records using a formula string, equivalent to a SQL `WHERE` clause